

When Names Matter and When They Don't: Variable Roles and Language Structure in Transformer Code Models

Anonymous authors

Paper under double-blind review

Abstract

Transformer code models generate fluent programs, but it remains unclear whether they track variables as semantic entities rather than local token patterns. We study *variable role representations* in controlled multilingual code, probing whether hidden states encode roles such as accumulator, iterator, index, boolean flag, and class/structure reference across naming and language changes. Across Qwen2.5 and Qwen2.5-Coder, linear probes recover these roles with high accuracy (macro-F1 up to 0.98), revealing robust role-selective directions in the residual stream. Our results uncover a sharp dissociation between lexical and structural cues: accumulator representations remain stable and often improve under misleading renaming, while index representations degrade substantially, indicating qualitatively different underlying mechanisms. Cross-language transfer further shows that some variable roles form language-agnostic representations, while others remain partially tied to language-specific syntax. Beyond variable roles, we show that language identity itself occupies a low-dimensional geometric subspace early in the model. Together, these findings provide evidence that transformer code models encode reusable program-level abstractions beyond syntax, offering a foundation for deeper mechanistic interpretability of code intelligence.

1 Introduction

Large language models trained on source code have become central tools for code completion, program synthesis, and bug fixing (Feng et al., 2020), yet whether their success reflects memorized surface patterns, learned syntactic structure, or genuine tracking of semantic entities such as variables and control flow remains unresolved (Karmakar & Robbes, 2021; Sontakke et al., 2022). This distinction matters especially for code, where small changes in variable binding can alter program behavior without changing the surrounding syntax.

This paper asks whether transformer code models represent variables by their *program role* rather than only by token identity. Accumulators, indices, iterators, boolean guards, and class/structure references can share surface forms while serving different functions, or preserve their function under renaming. Prior work shows that code models encode syntax and identifiers but struggle with deeper semantic properties (Troshin & Chirkova, 2022; Wan et al., 2022; Hooda et al., 2024), while recent mechanistic studies localize programming concepts and sparse features (Kargaran et al., 2025; Tahimic & Cheng, 2025; Yin et al., 2026). We probe controlled multilingual programs with known roles and counterfactual variants to identify when hidden states contain role-level machinery that generalizes across names and languages, and apply PCA to residual activations across seven languages to ask whether language identity itself is geometrically structured in the model’s internal representations.

2 Related Work

Prior work shows that code models encode substantial syntactic and identifier information, with CodeSearchNet and GraphCodeBERT establishing benchmarks and data-flow-aware representations (Husain et al., 2019; Guo et al., 2021). However, probing and counterfactual

studies find weaker evidence for robust semantic equivalence, data-flow, and control-flow understanding (Wan et al., 2022; Troshin & Chirkova, 2022; Hooda et al., 2024; North et al., 2025).

Recent interpretability work localizes programming concepts, code-correctness features, and neuron-level explanations in code LLMs (Kargaran et al., 2025; Tahimic & Cheng, 2025; Yin et al., 2026). We build on this line by treating variable roles as fine-grained, multilingual, and counterfactual test cases for program-state representation.

3 Experimental Setup

We evaluate variable-role representations in frozen **Qwen2.5-1.5B** activations using **CodeSearchNet** for realistic functions and **XLCoST/MuST-CoST** for aligned multilingual programs (Husain et al., 2019; Zhu et al., 2022a;b). XLCoST provides parallel implementations of competitive-programming problems across seven languages—C, C#, C++, Java, JavaScript, PHP, and Python—where the same algorithm appears in every language, making it well-suited for cross-lingual comparison of role representations. MuST-CoST extends this with additional multilingual snippet-level alignments. We label variable occurrences with lightweight static analysis plus manual audit, apply controlled renaming and role-swapping variants, and train layer-wise linear probes on hidden states.

We study five roles: **accumulators** aggregate values across loop iterations; **indices** serve as counters or keys; **iterators** are loop-bound sequence variables; **boolean controls** govern conditions, loops, returns, or indexing contexts; and **class/structure references** name declared types and their later uses. These roles span structural, lexical, single-language, and cross-language signals, which we evaluate through decodability, renaming robustness, and transfer. We also apply PCA to last-token residual activations across all seven XLCoST languages to ask a broader question: does the model organize programs by language identity in its residual stream, and if so, does that structure confound the role representations we measure?

4 Evidence for Variable Role Representations

We identify variable roles through static analysis of program ASTs, train layer-wise linear probes on frozen residual-stream activations, and assess each representation along three axes: linear decodability across layers, invariance to variable renaming, and transfer across programming languages.

4.1 Accumulator Variables

For accumulator variables, a layer-wise probe on Qwen2.5-1.5B residual activations peaks at $F1 = 0.893$ on XLCoST Python and transfers to compiled languages with 0.943-0.948 accuracy (Table 2, Figure 22). Renaming improves rather than hurts performance—misleading index-style names yield $\Delta F1 = +0.079$ —indicating that accumulator detection is driven more by structural update context than lexical identity (Table 1, Figures 20-21).

4.2 Index Variables

For index and key variables, probes peak at $F1 = 0.959$ for Qwen2.5-1.5B and 0.945 for DeepSeek-Coder-1.3B, exceeding accumulator performance because index names are lexically regular (Figure 23). Misleading renaming causes the largest drop of any role ($\Delta F1 = -0.272$ and -0.285), while cross-language transfer remains strong, showing that index representations combine strong lexical dependence with regular loop-index syntax (Table 3, Figures 24-25).

86 4.3 Iterator Variables

87 For iterator variables, a Qwen2.5-Coder-1.5B probe on XLCoST Python peaks at $F1 = 0.940$
 88 at layers 9 and 13 after rising sharply by layers 3–4 (Figure 3; best layer results in Table 6).
 89 The early plateau suggests a stable iterator representation that is more structural than
 90 token-identity based.

91 4.4 Boolean Variables

92 For boolean-control variables, occurrence type is strongly decodable from Qwen2.5-1.5B
 93 residual activations, with peak macro-F1 of 0.980 for Python, 0.971 for Java, and 0.890 for
 94 Go (Table 4). This indicates that the model represents how a flag is used in context, not only
 95 whether a token looks boolean.

96 4.5 Class and Structure References

97 For class and structure references, Qwen2.5-Coder-1.5B probes peak at macro-F1 0.952 for
 98 C++ and 0.949 for C#, with near-diagonal confusion matrices at the best layers (Figures 17–
 99 18). A token-identity control remains below the probe, giving baseline selectivity of 0.19–0.21
 100 and indicating that type-reference role is encoded contextually as well as lexically (Figure 14).
 101 Naming perturbations clarify this interpretation (Table 5): raw F1 rises to near ceiling
 102 under all-same or misleading names, but selectivity collapses or shrinks, showing that
 103 those gains are mostly identity-driven. Cross-lingual transfer remains above baseline but
 104 asymmetric, suggesting a partially shared type-reference direction with deeper language-
 105 specific structure (Figure 16).

106 5 Structural-Class and Identifier-Role Probing

107 As a syntactic baseline, we probe whether residual activations encode token classes and
 108 identifier roles. Structural class is nearly linearly decodable by early layers in both Qwen2.5-
 109 1.5B and Qwen2.5-Coder-1.5B: within-language macro-F1 reaches ≈ 0.97 – 0.99 by layer 2,
 110 while cross-lingual transfer plateaus around 0.85–0.90 (Figures 4–6). The Coder model
 111 exposes this information earlier, but the ceiling is similar across models.

112 Token-identity controls show that much of this signal is lexical: control F1 is high at the
 113 embedding layer, while selectivity grows only modestly with depth (Figure 11). Identifier-
 114 role probes also reach near-ceiling within-language performance, but cross-lingual transfer
 115 is asymmetric, suggesting that syntactic identifier roles are less language-invariant than the
 116 strongest variable-role directions above (Figure 13).

117 6 Language-Specific Structure in Residual Representations

118 Complementing the variable-role probes of Section 4, we ask whether Qwen2.5-Coder-1.5B
 119 encodes programming language identity as a structured geometric signal in its residual
 120 stream. We extract last-token residual activations across all 29 layers for 19,471 XLCoST
 121 programs and apply PCA independently at each layer, coloring points by language and by
 122 static/dynamic typing bucket—C, C#, C++, and Java as statically typed; JavaScript, PHP,
 123 and Python as dynamically typed. Language separation emerges from the embedding layer
 124 onward ($r = +0.883$ at layer 0) and peaks at layer 4, where tight, well-isolated per-language
 125 clusters appear and the static and dynamic groups occupy opposite ends of PC1 ($r = +0.941$,
 126 EVR = 37%). The scree plot at layer 4 (Figure 19) shows a sharp elbow after PC3 (PC2: 22%,
 127 PC3: 11%), confirming that language-identity variance concentrates in a low-dimensional
 128 subspace. Separation holds through layers 8–14 ($r \approx 0.90$), undergoes a sign flip at layer 20
 129 ($r = -0.903$, a refit artifact), and degrades at layer 28 ($r = -0.576$) as clusters intermix.

130 The 3D PCA (Figure 1) clarifies structure compressed out of the 2D projection. At lay-
 131 ers 8 and 14, what appear as elongated clusters in 2D resolve into tight, needle-like linear

manifolds per language in 3D, indicating that within-language variation is structured and low-dimensional rather than isotropic. The static/dynamic partition remains cleanly separated across all three principal components through layer 20, and the breakdown at layer 28 is more apparent in 3D, where the two groups collapse into a shared region that the 2D projection partially obscures.

Per-language visualization (Figure 2) confirms that the static/dynamic axis reflects genuine per-language clustering rather than a coarser two-group split. Each language occupies a distinct, coherent region stable from layers 4 through 20, indicating that the model encodes language identity at the individual-language level rather than only at the static/dynamic grouping. On the static side, C and C++ cluster almost on top of each other—both share low-level constructs and in competitive-programming contexts their code is nearly identical token-by-token—while Java and C# form an equally tight pair, consistent with their syntactic proximity. On the dynamic side the structure is looser: PHP sits in its own isolated cluster gravitating toward Python and JavaScript, which are co-located but more dispersed than the static pairs, reflecting their shared dynamic nature alongside substantial syntactic divergence. The static/dynamic separation is therefore a geometric consequence of how these seven clusters arrange along PC1, not direct evidence that the model has internalized type discipline as an abstract concept.

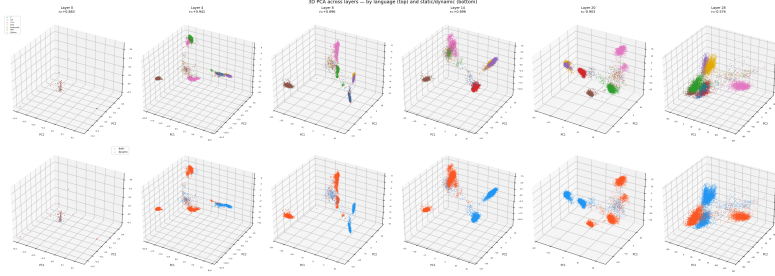


Figure 1: 3D PCA of last-token activations across layers, colored by language and static/dynamic type group.

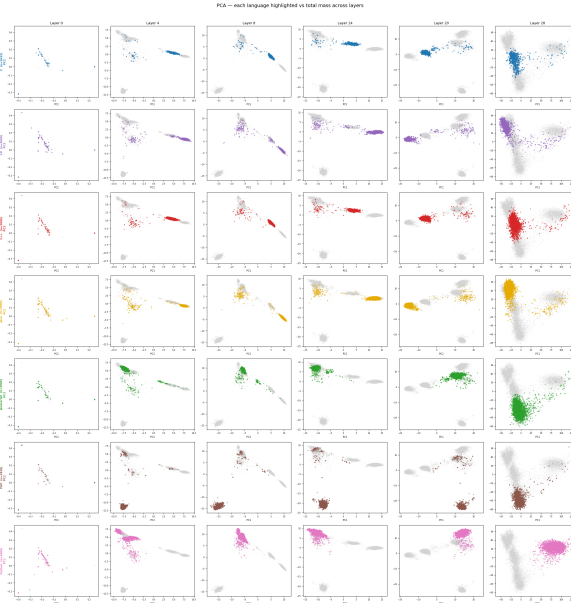


Figure 2: Per-language PCA clusters across layers, highlighting stable language-identity structure.

7 Conclusion and Future Work

We have presented a mechanistic probing study of variable role representations and syntactic structure in Qwen2.5-1.5B and Qwen2.5-Coder-1.5B across multiple programming languages. Linear probes on the residual stream reliably decode all five variable roles: accumulator, index, iterator, boolean, and class reference with macro-F1 of 0.85–0.99: iterator representations stabilize at $F1 \approx 0.93$ –0.94 by layers 8-9, boolean usage context is decodable at ≈ 0.9 across Python, Java, and Go, and class references reach ≈ 0.93 –0.95 by layer 3-4 with positive selectivity (0.15-0.20) confirming signal beyond token identity. The most striking role-level finding is a dissociation in cue type: index detection degrades sharply under misleading names ($\Delta F1 = -0.272$ for Qwen2.5-1.5B, -0.285 for DeepSeek-Coder-1.3B) while accumulator detection *improves* ($+0.079$ F1), revealing qualitatively different mechanisms for different roles. At the syntactic level, coarse token class is near-perfectly decodable by layer 2 (macro-F1 ≈ 0.97 –0.99), though a Hewitt control shows most of this is token-identity memorization (selectivity only 0.20-0.27); code pretraining shifts the resolution layer earlier ($L8 \rightarrow L3$) without raising the ceiling, and identifier-role cross-lingual transfer degrades to 0.58-0.65 in Coder’s late layers. Finally, PCA of last-token activations shows that language identity is geometrically structured from layer 0 onwards, peaking at layer 4 ($r = +0.941$, PC1 accounting for 37% of variance), with 3D projections revealing tight within-language manifolds and interpretable sub-groupings such as C/C++ and Java/C# pairs. All probing results are correlational; future work should establish causal relevance via activation patching and sparse autoencoder analysis, and extend coverage to larger models and the remaining accumulator cross-language conditions.

References

- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.139. URL <https://aclanthology.org/2020.findings-emnlp.139/>.
- Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=jLoC4ez43PZ>.
- Ashish Hooda, Mihai Christodorescu, Miltiadis Allamanis, Aaron Wilson, Kassem Fawaz, and Somesh Jha. Do large code models understand programming concepts? counterfactual analysis for code predicates. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltos Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. Technical Report MSR-TR-2019-45, Microsoft, September 2019.
- Amir Hossein Kargaran, Yihong Liu, François Yvon, and Hinrich Schütze. How programming concepts and neurons are shared in code language models, 2025. URL <https://arxiv.org/abs/2506.01074>.
- Anjan Karmakar and Romain Robbes. What do pre-trained code models know about code? In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1332–1336, 2021. doi: 10.1109/ASE51524.2021.9678927. URL https://www.researchgate.net/publication/354141609_What_do_pre-trained_code_models_know_about_code.
- Marc North, Amir Atapour-Abarghouei, and Nelly Bencomo. Beyond syntax: How do llms understand code? In *2025 IEEE/ACM 47th International Conference on Software*

- 201 *Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pp. 86–90, 2025. doi: 10.1109/
202 ICSE-NIER66352.2025.00023.
- 203 Ankita Nandkishor Sontakke, Manasi Patwardhan, Lovekesh Vig, Raveendra Kumar
204 Medicherla, Ravindra Naik, and Gautam Shroff. Code summarization: Do transform-
205 ers really understand code?, 2022. URL <https://openreview.net/forum?id=rI5l12-1Zc>.
206 DL4C 2022.
- 207 Kriz Tahimic and Charibeth Cheng. Mechanistic interpretability of code correctness in llms
208 via sparse autoencoders, 2025. URL <https://arxiv.org/abs/2510.02917>.
- 209 Sergey Troshin and Nadezhda Chirkova. Probing pretrained models of source codes. In
210 Jasmijn Bastings, Yonatan Belinkov, Yanai Elazar, Dieuwke Hupkes, Naomi Saphra, and
211 Sarah Wiegrefe (eds.), *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and*
212 *Interpreting Neural Networks for NLP*, pp. 371–383, Abu Dhabi, United Arab Emirates
213 (Hybrid), December 2022. Association for Computational Linguistics. doi: 10.18653/v1/
214 2022.blackboxnlp-1.31. URL <https://aclanthology.org/2022.blackboxnlp-1.31/>.
- 215 Yao Wan, Wei Zhao, Hongyu Zhang, Yulei Sui, Guandong Xu, and Hai Jin. What do
216 they capture? a structural analysis of pre-trained language models for source code.
217 In *Proceedings of the 44th International Conference on Software Engineering, ICSE ’22*, pp.
218 2377–2388, New York, NY, USA, 2022. Association for Computing Machinery. ISBN
219 9781450392211. doi: 10.1145/3510003.3510050. URL [https://doi.org/10.1145/3510003.](https://doi.org/10.1145/3510003.3510050)
220 [3510050](https://doi.org/10.1145/3510003.3510050).
- 221 Zhe Yin, Xiaodong Gu, and Beijun Shen. Neuron-guided interpretation of code llms: Where,
222 why, and how?, 2026. URL <https://arxiv.org/abs/2512.19980>.
- 223 Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chan-
224 dan K. Reddy. Xlcost: A benchmark dataset for cross-lingual code intelligence, 2022a.
225 URL <https://arxiv.org/abs/2206.08474>.
- 226 Ming Zhu, Karthik Suresh, and Chandan K. Reddy. Multilingual code snippets training
227 for program translation. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
228 volume 36, pp. 11783–11790, 2022b. doi: 10.1609/aaai.v36i10.21434. URL [https://doi.](https://doi.org/10.1609/aaai.v36i10.21434)
229 [org/10.1609/aaai.v36i10.21434](https://doi.org/10.1609/aaai.v36i10.21434).

230 A Appendix Figures

Table 1: Accumulator probe results under six naming strategies, Python XLCoST, Qwen2.5-1.5B.

Strategy	Best Layer	Test Acc	Test F1	Δ F1
Baseline (original names)	L9	0.984	0.893	—
Random nouns	L10	0.979	0.870	−0.023
Single chars (a, b, c. . .)	L9	0.971	0.840	−0.053
All same (x)	L5	0.986	0.969	+0.076
Numeric (v1, v2. . .)	L3	0.999	0.986	+0.093
Misleading (index-style names)	L9	0.996	0.972	+0.079

Table 2: Cross-language accumulator probe. Python-trained probe (L9) applied to four additional languages.

Language	Programs	In-Domain F1	Cross-Lang Acc
Python	181 / 500	0.893 (L9)	—
Java	208 / 300	0.952 (L9)	0.948
C++	204 / 300	0.939 (L7)	0.948
C	165 / 300	0.970 (L11)	0.943
C#	207 / 300	0.950 (L9)	0.945

Table 3: Index/key probe F1 under key naming conditions across models, and comparison with the accumulator role (Qwen2.5-1.5B). Both models show the largest drop under misleading renaming, opposite to the accumulator pattern.

Condition	Model	Best Layer	Test F1	Δ F1
<i>Index role</i>				
Baseline	Qwen2.5-1.5B	L8	0.959	—
Baseline	DeepSeek-Coder-1.3B	L11	0.945	—
All same (x)	Qwen2.5-1.5B	L14	0.742	−0.217
Misleading	Qwen2.5-1.5B	L15	0.687	−0.272
Misleading	DeepSeek-Coder-1.3B	L16	0.660	−0.285
<i>Accumulator role (Qwen2.5-1.5B), for contrast</i>				
Baseline	Qwen2.5-1.5B	L9	0.893	—
Misleading	Qwen2.5-1.5B	L9	0.972	+0.079

Language	Layer 0 acc	Layer 0 F1	Peak Layer	Peak F1	Δ F1
Python	0.702	0.428	14	0.980	0.552
Java	0.908	0.652	17	0.971	0.319
Go	0.992	0.886	18	0.890	0.004

Table 4: Layer-wise decodability of boolean occurrence_type in Qwen2.5-1.5B across CodeSearchNet languages. Probes are trained on mean-pooled residual activations from boolean-flag variable sites (Python $n = 7,981$; Java $n = 7,995$; Go $n = 7,987$) using an 80/20 stratified split. Layer 0 columns report embedding-layer accuracy and macro F1; Peak Layer and Peak F1 report the best held-out macro F1; Δ F1 is the improvement over Layer 0 F1.

Condition	Best Layer	Test F1	Δ F1
<i>C++ type def→use</i>			
Baseline (original names)	L7	0.952	—
All same (T)	L5	1.000	+0.048
Misleading (variable-style)	L8	0.999	+0.047
<i>C# type def→use</i>			
Baseline (original names)	L19	0.949	—
All same (T)	L12	0.998	+0.049
Misleading (variable-style)	L7	0.999	+0.050

Table 5: Class/struct type definition→use probe performance under type-naming conditions (XLCoST/MuST-CoST, Qwen2.5-Coder-1.5B). *All same* collapses every declared type to a single token, while *misleading* renames types to distinct variable-style names. The table reports the best probe layer, held-out F1, and F1 change relative to the original-name baseline for each language.

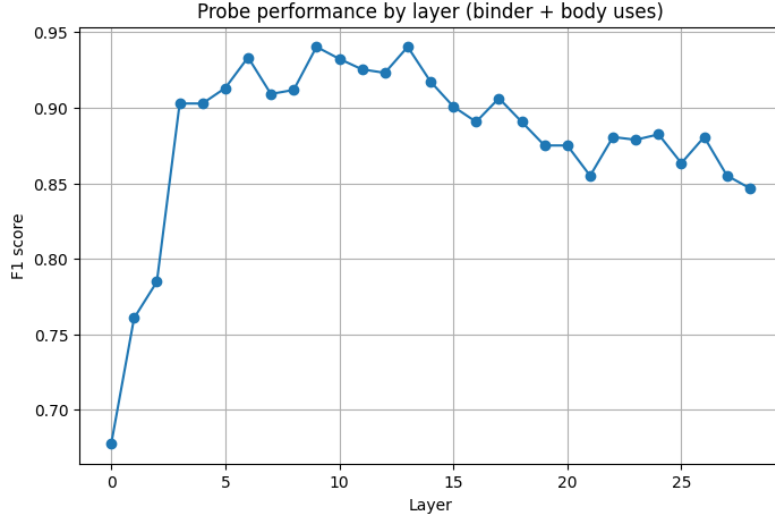


Figure 3: Iterator variable probe F1 vs. layer for Python XLCoST programs, Qwen2.5-Coder-1.5B. Labels cover both the for-loop binder and all scope-tracked body uses of the iterator variable.

Table 6: Iterator variable probe summary, Python XLCoST programs. Δ F1 is improvement over the embedding layer (L0).

Model	Best Layer	Test F1	Δ F1
Qwen2.5-Coder-1.5B (L0)	L0	0.678	—
Qwen2.5-Coder-1.5B	L9	0.940	+0.262

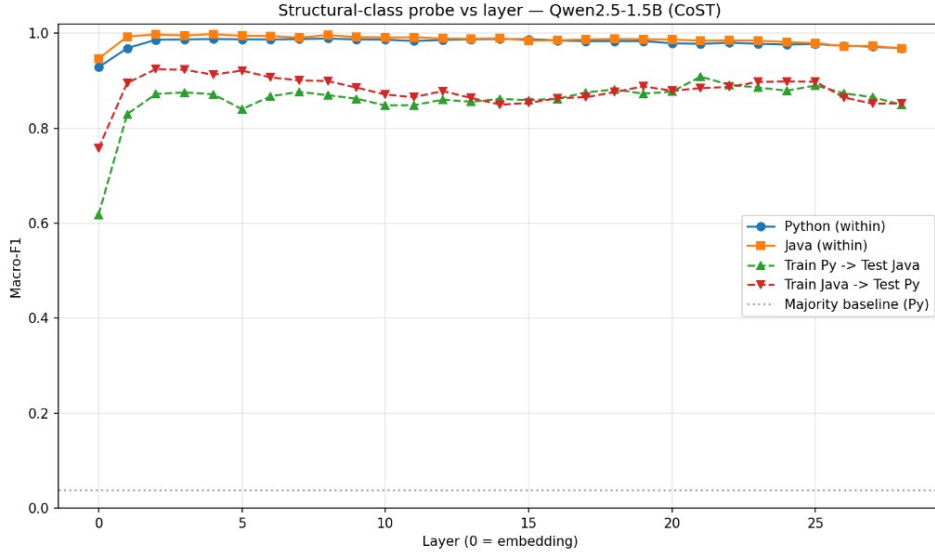


Figure 4: Structural-class probe macro-F1 vs. layer, Qwen2.5-1.5B, CoST corpus. Solid lines: within-language Python and Java. Dashed lines: cross-lingual transfer. Dotted line: majority baseline.

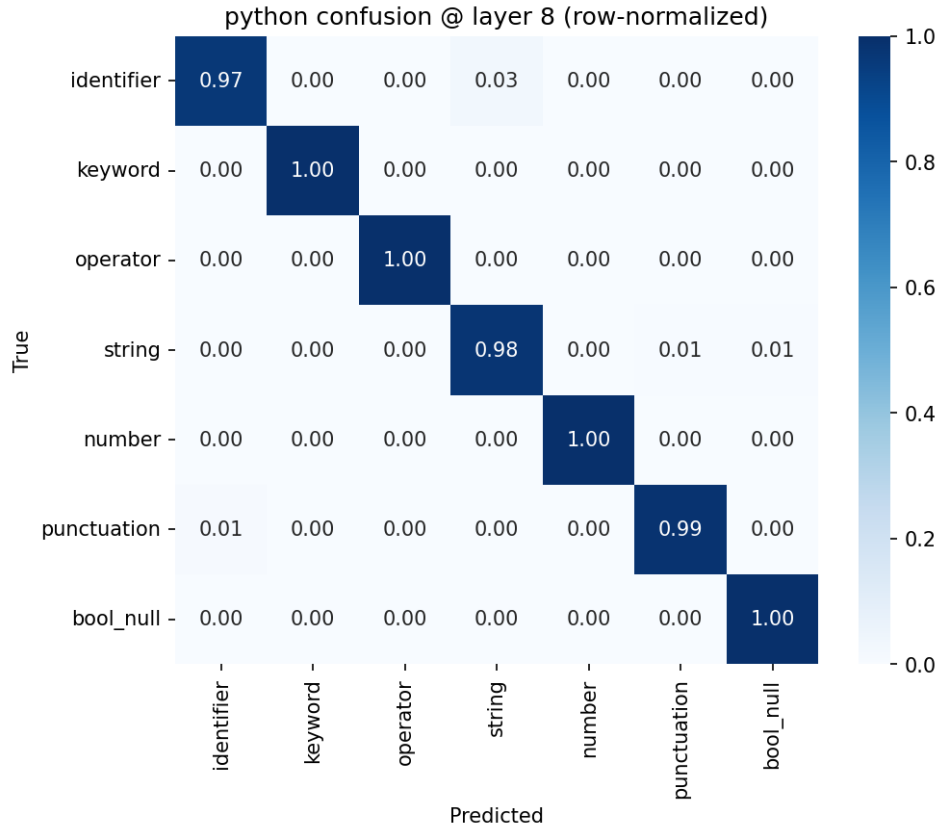


Figure 5: Python structural-class confusion matrix, Qwen2.5-1.5B, layer 8, row-normalized.

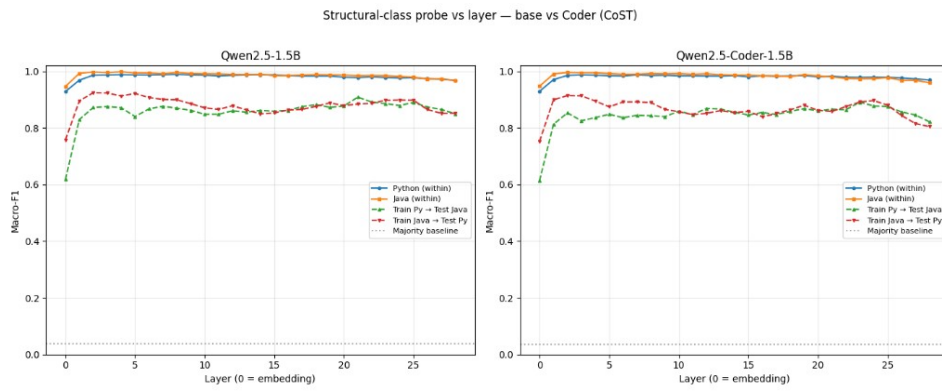


Figure 6: Structural-class probe macro-F1 vs. layer, Qwen2.5-1.5B on the left and Qwen2.5-Coder-1.5B on the right, CoST corpus.

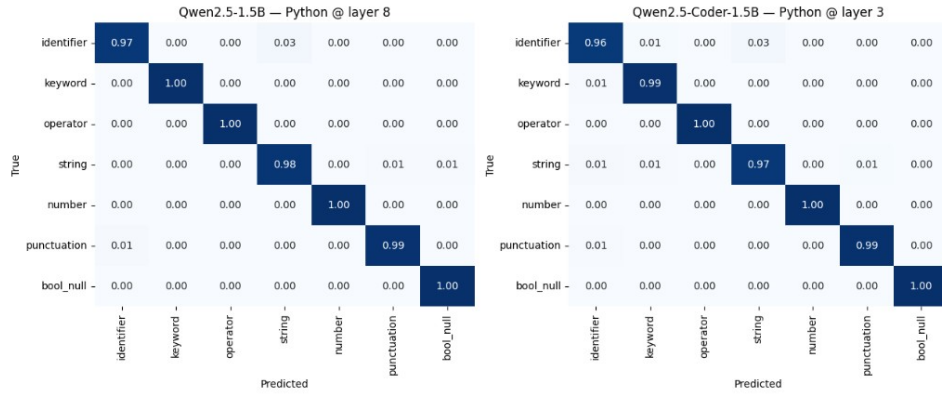


Figure 7: Python structural-class confusion matrices at each model’s peak layer: Qwen2.5-1.5B at layer 8 on the left vs. Qwen2.5-Coder-1.5B at layer 3 on the right.

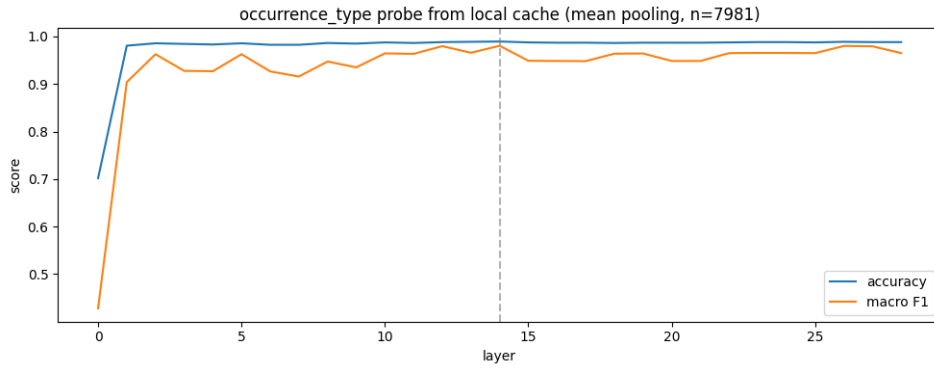


Figure 8: Boolean variable usage probe results for Python CodeSearchNet snippets using Qwen2.5-1.5B residual activations.

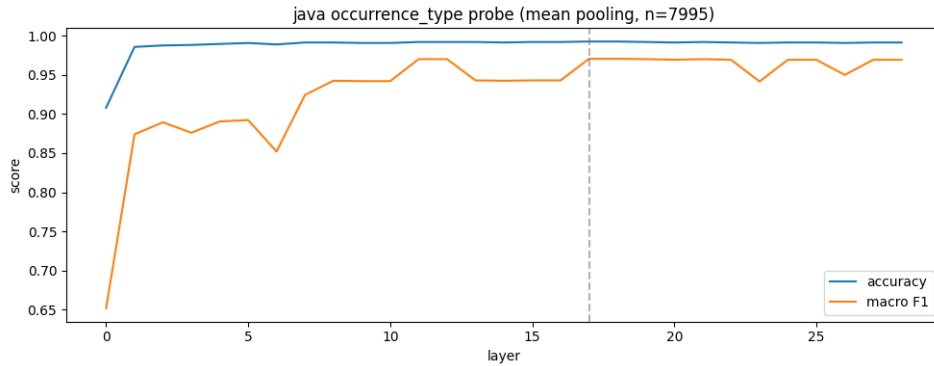


Figure 9: Boolean variable usage probe results for Java CodeSearchNet snippets using Qwen2.5-1.5B residual activations.

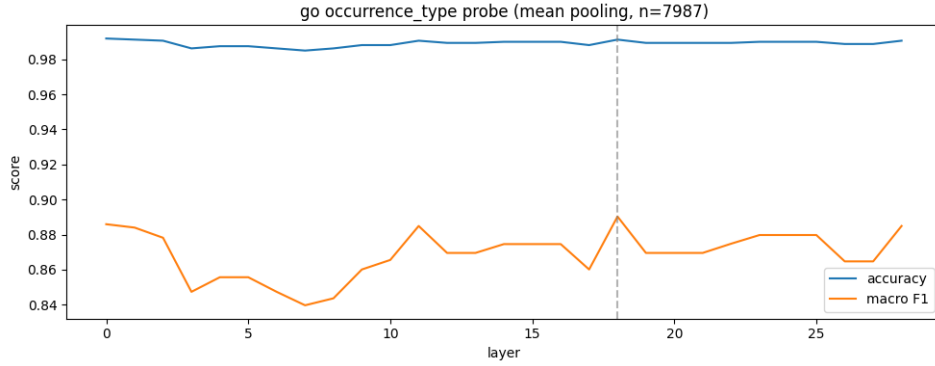


Figure 10: Boolean variable usage probe results for Go CodeSearchNet snippets using Qwen2.5-1.5B residual activations.

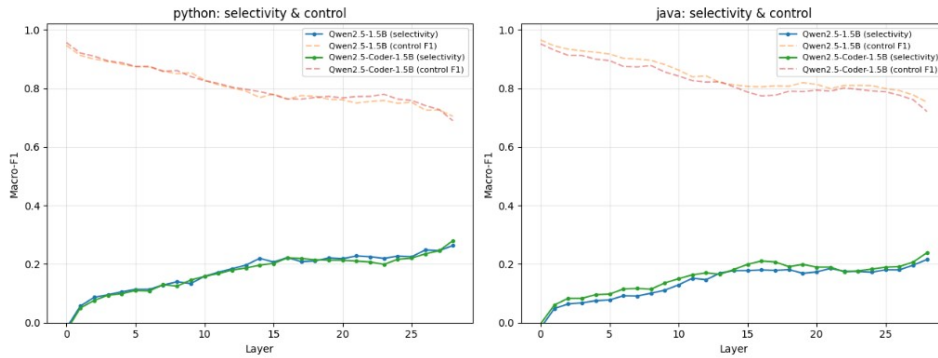


Figure 11: Selectivity, shown as solid lines, and raw control-task F1, shown as dashed lines, vs. layer for Python on the left and Java on the right, comparing Qwen2.5-1.5B and Qwen2.5-Coder-1.5B.

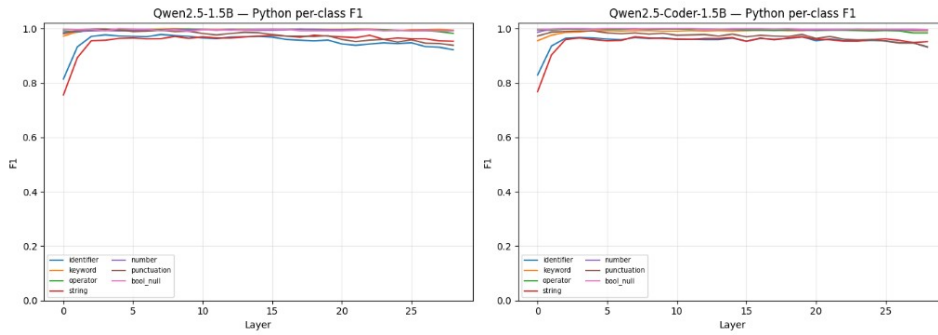


Figure 12: Per-class F1 vs. layer for Python, Qwen2.5-1.5B on the left vs. Qwen2.5-Coder-1.5B on the right.

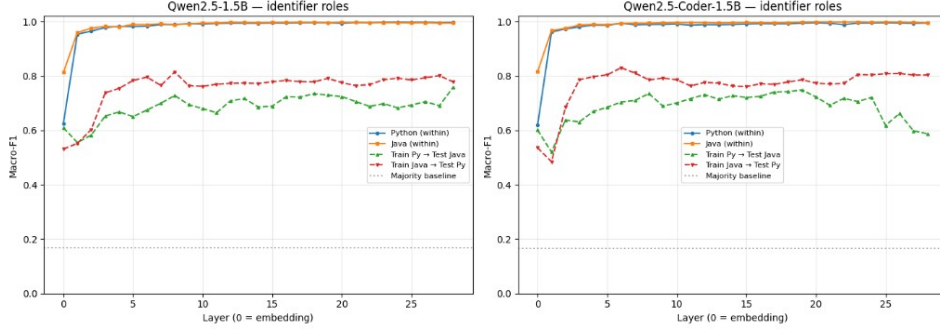


Figure 13: Identifier-role probe macro-F1 vs. layer, Qwen2.5-1.5B on the left vs. Qwen2.5-Coder-1.5B on the right. Here identifier role denotes syntactic categories such as variable name versus function name, not semantic variable roles.

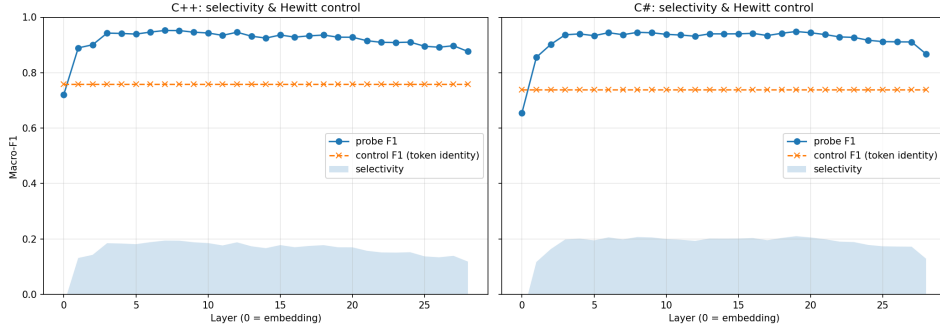


Figure 14: Selectivity (shaded fill) and raw token-identity control-task F1 (dashed) vs. layer for the class/struct type definition→use probe, C++ (left) and C# (right), Qwen2.5-Coder-1.5B. Selectivity (probe macro-F1 minus control) is positive and stable across depth (≈ 0.19 – 0.21 at peak), indicating the type-reference signal is not explained by token identity alone. This is the baseline (un-renamed) condition; selectivity under the naming perturbations of Table 5 is reported separately.

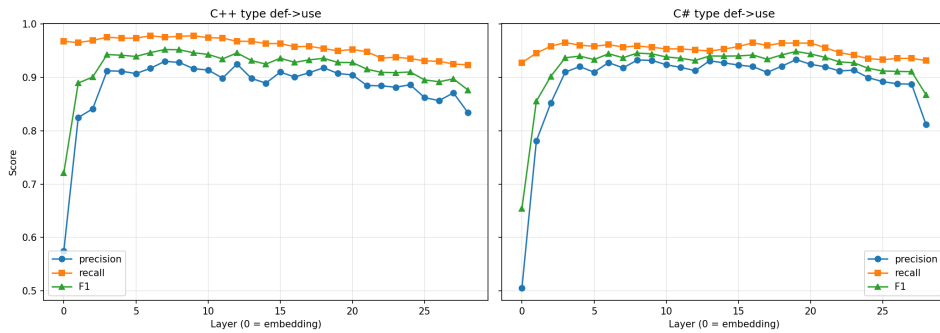


Figure 15: Per-metric class/struct type definition-to-use probe results vs. layer using Qwen2.5-Coder-1.5B residual activations, C++ on the left and C# on the right. Solid lines: precision, recall, and macro-F1.

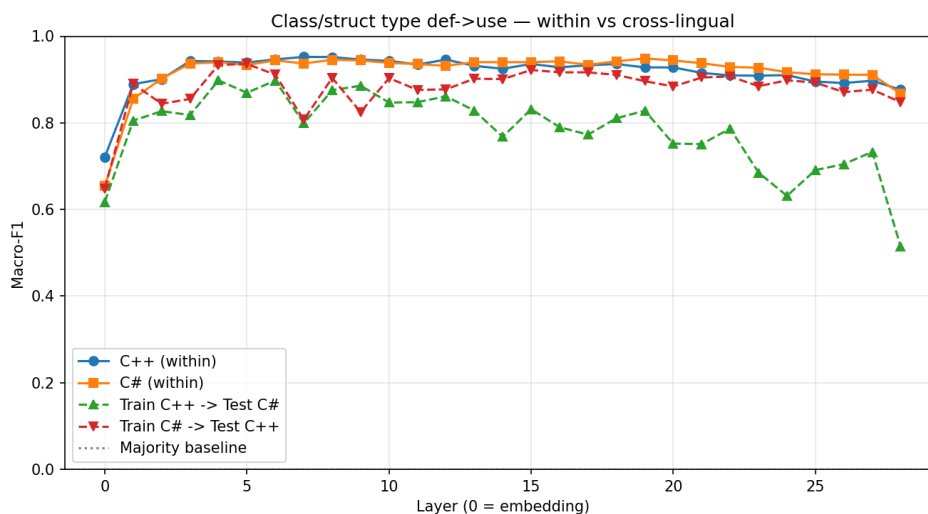


Figure 16: ... Dotted line: majority baseline. Transfer is asymmetric: Train C# $\rightarrow$ Test C++ tracks the within-language curves, while Train C++ $\rightarrow$ Test C# degrades in deeper layers (to ≈ 0.51).

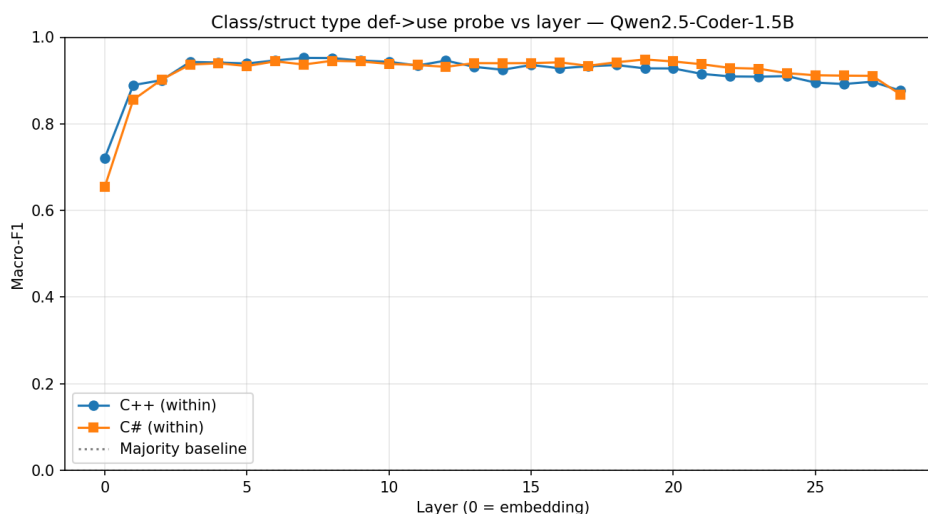


Figure 17: Class/struct type definition-to-use probe macro-F1 vs. layer for Qwen2.5-Coder-1.5B, C++ and C# within-language. Dotted line: majority baseline.

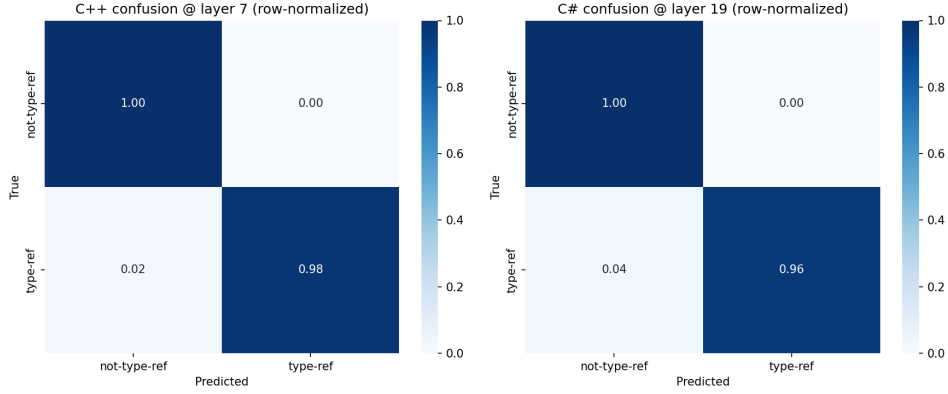


Figure 18: Class/struct type definition-to-use confusion matrices at each language’s peak layer (row-normalized): C++ at layer 7 on the left and C# at layer 19 on the right, Qwen2.5-Coder-1.5B.

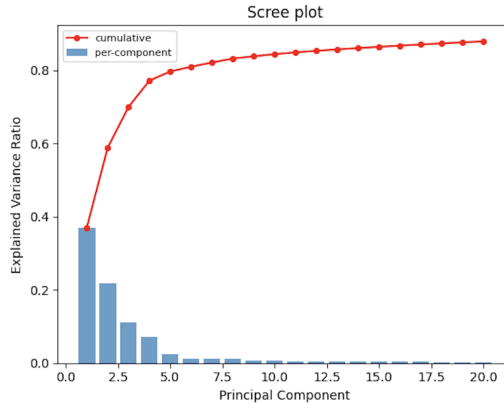


Figure 19: Scree plot of PCA at layer 4, Qwen2.5-Coder-1.5B last-token activations. Bars show per-component explained variance ratio; the red curve shows the cumulative total. The sharp elbow after PC3 indicates that language-identity variance is concentrated in a low-dimensional subspace.

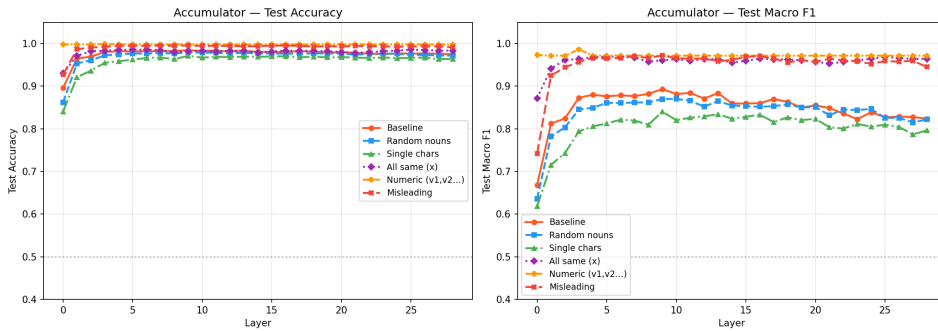


Figure 20: Accumulator variable probe accuracy and macro-F1 vs. layer for Python XLCOST programs, Qwen2.5-1.5B, across six variable-naming strategies. The baseline peaks at $F1 = 0.893$ (L9); all-same and misleading-name strategies exceed the baseline, confirming that structural context dominates lexical cues for this role.

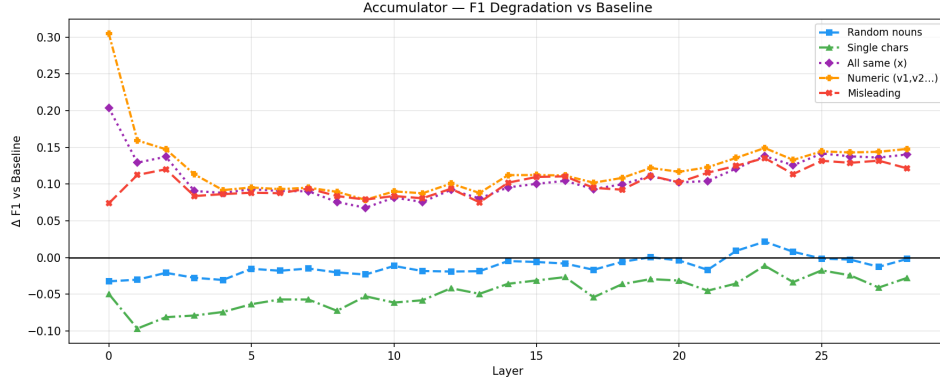


Figure 21: Δ macro-F1 relative to baseline naming for each perturbation strategy, accumulator role. Positive bars (all-same +0.076, numeric +0.093, misleading +0.079) indicate that removing or scrambling variable names *improves* accumulator detection, in sharp contrast to the index role where misleading names cause $\Delta F1 = -0.272$.

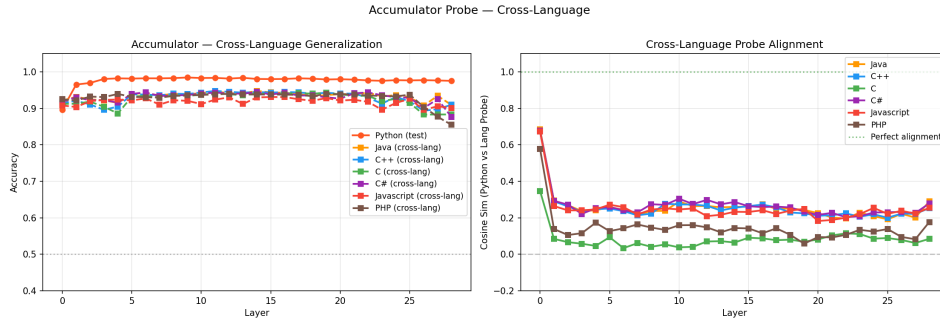


Figure 22: Cross-language transfer for the accumulator probe. A Python-trained probe (L9) achieves 0.943–0.948 accuracy on Java, C++, C, and C#, indicating a language-agnostic accumulator direction in the residual stream. In-domain F1 for compiled languages (0.939–0.970) exceeds Python (0.893) due to the syntactic regularity of += in C-family loop bodies.

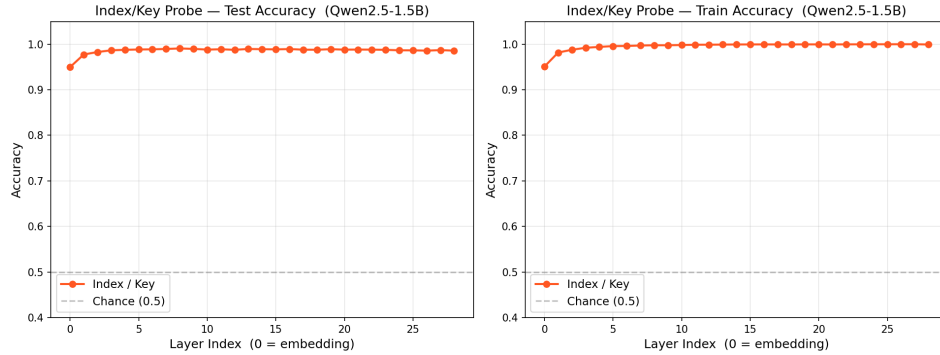


Figure 23: Index/key variable probe accuracy and macro-F1 vs. layer, Python XLCoST, Qwen2.5-1.5B, baseline (original variable names). Peak F1 = 0.959 at L8 exceeds the accumulator role, reflecting the strong lexical regularity of index-style names (i, j, idx).

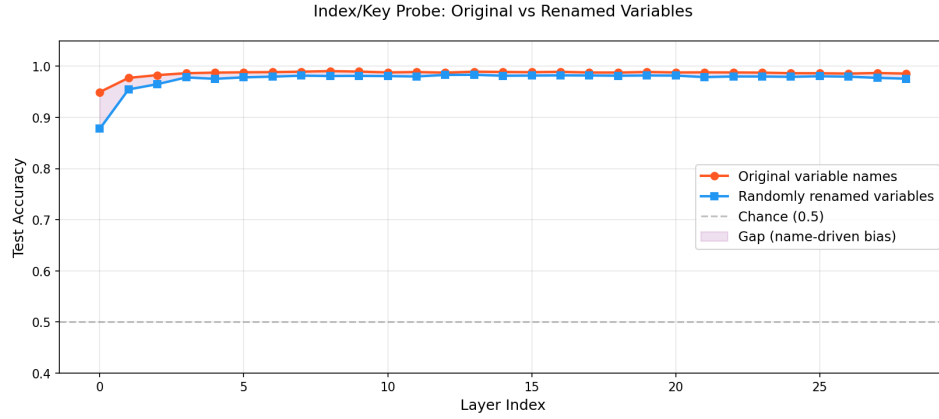


Figure 24: Index/key probe F1 under original versus misleading variable names. Misleading renaming causes $\Delta F1 = -0.272$, the largest drop of any role studied, indicating that index role detection depends heavily on surface lexical identity.

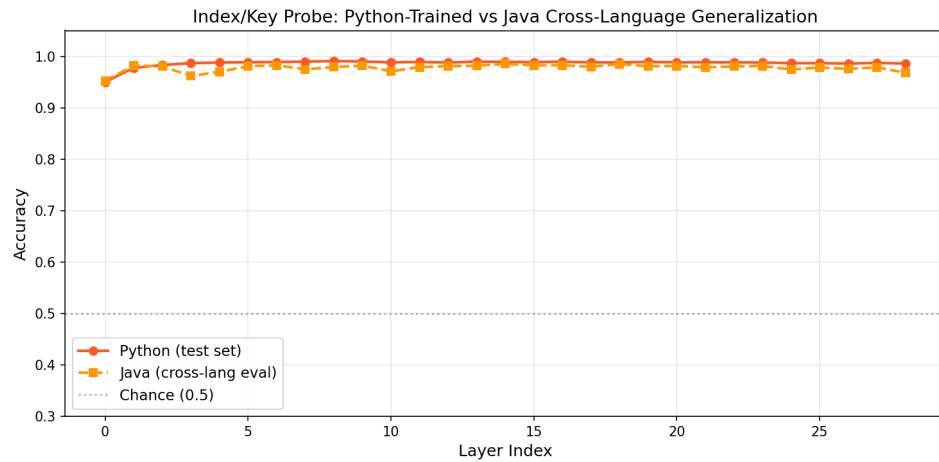


Figure 25: Cross-language transfer for the index/key probe. A Python-trained probe achieves 0.973–0.988 accuracy on Java and C++, reflecting the syntactic regularity of loop-index patterns across languages and exceeding the accumulator transfer range (0.943–0.948).